



Agentic eXecution & Oversight Network

SYSTEM DOCUMENTATION & TECHNICAL MANUAL

Enterprise Multi-Agent Control Tower, Telemetry Observability & Governance Platform

Platform Version: 1.0.0

Release Status: Production Stable

Security Classification: Confidential

Authors: Hackathon Development Team

Table of Contents

This technical manual details the setup, operation, database layouts, and governance structures of the AXON. system:

CHAPTER SCHEME	PAGE NO.
CHAPTER 1: INTRODUCTION	03 - 04
1.1 Introduction	03
1.2 Existing System	03
1.3 Needs and Scope of Computer System	04
CHAPTER 2: PROPOSED SYSTEM	04 - 05
2.1 Objective	04
2.2 Requirement Engineering	05
2.3 Requirement Gathering	05
CHAPTER 3: SYSTEM ANALYSIS (DIAGRAMS OMITTED BY REQUEST)	05
CHAPTER 4: SYSTEM DESIGN	06 - 08
4.1 Database Design	06
4.2 Input Design and It's Samples	07
4.3 Output Design (on screen)	08

CHAPTER 5: IMPLEMENTATION	09
5.1 Installation Process	09
5.2 User Guideline	09
CHAPTER 6: REPORTS	10
CHAPTER 7: CONCLUSION, LIMITATIONS AND SUGGESTIONS	10
CHAPTER 8: REFERENCES	10

Chapter 1: Introduction

1.1 Introduction

With the exponential growth of Autonomous Multi-Agent AI networks in the enterprise sector, monitoring and controlling their execution flows, budget spend limits, and security protocols has become a critical operational necessity. AI Agents operate semi-autonomously using LLMs, generating structured logs, reasoning steps, and API database invocations.

AXON. (Agentic eXecution & Oversight Network) is an enterprise control tower developed specifically to address these operational issues. It serves as a central nervous system to oversee multi-agent interactions, monitor LLM token budgets dynamically, prevent infinite execution loops, and implement manual approval gateways for high-risk operations.

1.2 Existing System

Currently, developers and system administrators manage AI agent execution blocks via raw stdout terminal logs or cloud-hosted logging applications. The drawbacks of these traditional systems include:

- **Zero Cost Safeguards:** Malfunctioning agents can enter infinite retry loops, repeating expensive queries and generating massive runaway API billing bills in minutes.
- **No Centralized Visual Lineage:** Complex orchestration routing is represented as dry text trails, making it incredibly difficult to trace communication loops between specialists.
- **Absence of Governance Gates:** High-risk database operations or payouts are executed automatically by the agents without checking with human manager authorizations.

1.3 Needs and Scope of Computer System

The AXON. platform introduces an observability and safety network layer, providing:

1. **Real-time Telemetry:** Visualizes dependencies, operational states, and latencies across all registered agents.
2. **Cost Caps & Throttling:** Calculates prompt and completion token costs in real-time to enforce spending thresholds.
3. **Loop Interceptor:** Breaks consecutive identical queries before they drain API resources.

4. **Human-in-the-Loop Gateway:** Safeguards corporate integrations by staging actions for review.

Chapter 2: Proposed System

2.1 Objective

The core objective of AXON. is to deploy a secure, responsive, and visually detailed command center overlay for multi-agent workflows. It provides operators with telemetry logs, call lineage maps, and spend analytics without requiring heavy modifications to existing agent codebases.

2.2 Requirement Engineering

The system requirements are modeled to optimize enterprise infrastructure compliance:

- **Functional Requirements:** Server-Sent Events (SSE) log streaming, cost calculator triggers, settings configurations panel, and action queue approvals.
- **Non-Functional Requirements:** Thread-safe SQLite backend transactions, sub-second latency rendering, and clean responsive CSS dashboard grids.

2.3 Requirement Gathering

Interviews with DevOps and AI engineers indicated that runaway token costs and lack of manual validation gates are the primary barriers to deploying autonomous agent fleets. AXON.'s loop breaker controls and governance queues directly resolve these friction points.

Chapter 3: System Analysis

Note: Detailed System Analysis diagrams (including DFD, ERD, and UML modeling) have been omitted from this technical manual by request. Relational database layouts and operational flows are detailed inside Chapter 4.

Chapter 4: System Design

4.1 Database Design

AXON. utilizes an SQLite database (backend/control_tower.db) with SQLAlchemy ORM mappings. Below are the key tables defining the system schemas:

Table: agents (Stores profiles and billing ceilings)

Column	Type	Description
id	VARCHAR (PK)	Unique agent string (e.g. 'finance_agent').
name	VARCHAR	Descriptive agent label.
cost_limit	FLOAT	Maximum allowed spend (USD).
status	VARCHAR	State: 'ACTIVE', 'SUSPENDED', or 'IDLE'.

Table: tasks (Logs user queries and execution states)

Column	Type	Description
id	VARCHAR (PK)	UUID for the run.
agent_id	VARCHAR (FK)	References agent executing the task.
status	VARCHAR	State: 'RUNNING', 'SUCCESS', 'FAILED', or 'PENDING_APPROVAL'.

Table: logs (Granular agent statements)

Column	Type	Description
id	INTEGER (PK-AI)	Log record sequence index.
task_id	VARCHAR (FK)	Associates statements to parent tasks.
level	VARCHAR	Log level: 'INFO', 'WARNING', or 'ERROR'.

4.2 Input Design and It's Samples

The system accepts inputs across three primary interfaces:

1. **User Prompts (Workspace Control):** Accepts text inputs and scenario configurations (e.g. Standard Prompt, HR Setup, Finance Approval, Retry Anomaly).

Sample: "Verify ledger transaction records and deploy AWS server using DevOps Engineer."

2. **Governance Approval Inputs:** Reviewers input comments and click action buttons to approve/reject staged operations.

Sample: "Approved. Verified AWS backup server configuration looks compliant."

Admin Drawer Configuration Sliders: Adjusts global variables:

- Simulation speed: 0.1s - 2.0s
- Max recursion loop breaker: 3 - 10 attempts
- Daily cost warning cap: \$10 - \$1000

4.3 Output Design (on screen)

AXON. presents telemetry output data on a single unified console screen:

- **Live DAG Lineage Map:** Renders a canvas layout of agent dependency nodes connected by communication arrows, streaming execution sequences live.
- **Per-Agent Spend Telemetry:** Visualizes agent cost capacities against their ceilings using animated progress bars. Spends exceeding limits automatically color the bars warning red.
- **Historical Spending Charts:** Plots Recharts area diagrams illustrating historical token consumes (filterable by all agents or individual agents).
- **Real-time Log Terminal:** Streams log statements from the background server with matching colors (INFO=white, WARNING=yellow, ERROR=red).

Chapter 5: Implementation

5.1 Installation Process

Follow these steps to deploy AXON. on your local machine:

Step 1: Set up the Backend Server

```
cd backend pip install -r requirements.txt python seed_data.py python main.py
```

Step 2: Launch the Web Console

```
cd frontend npm install npm run dev
```

5.2 User Guideline

1. Open <http://localhost:5173> in your web browser.
2. Choose a Scenario preset (e.g. Finance Approval Request) and click **Run Engine**.
3. During active runs, click the red **Stop Engine** button to abort tasks immediately.
4. Access settings using credentials (admin / admin123) to modify caps or wipe metrics.

Chapter 6: Reports

The system compiles and exports real-time reports based on database records:

- **Agent Financial Spend Report:** Tabulates cumulative prompt/completion token expenditures per agent category against their budgets.
- **System Health Score Report:** Auto-calculates overall success rates, pending approvals, and active loop alerts.
- **Operational Audit Log Report:** Chronological log sequences of agent collaborations exported for compliance reviews.

Chapter 7: Conclusion, Limitations and Suggestions

7.1 Conclusion

AXON. successfully bridges the gap between autonomous multi-agent systems and enterprise security protocols. It delivers real-time monitoring and safety control switches, keeping LLM spends secure.

7.2 Limitations

1. Runs on a localized SQLite engine, which requires migration to PostgreSQL for high-throughput production clusters.
2. Features a simulated agent engine; connection to live webhooks is required to run live actions on AWS/APIs.

7.3 Suggestions

Future modules should incorporate Slack/Email alert systems for cost limits breaches, and OAuth2 login authentication for settings control.

Chapter 8: References

- FastAPI Web Framework Documentation: <https://fastapi.tiangolo.com>
- Vite Tooling and React Core APIs: <https://vitejs.dev>

- SQLAlchemy ORM Documentation: <https://www.sqlalchemy.org>
- Material UI Component Library: <https://mui.com>